

Snapdragon

An offline, on-device accessibility co-pilot · github.com/challalokesh08/snapdragon

Short description

Snapdragon is an offline, on-device accessibility co-pilot for people who are blind, deaf or hard of hearing. It runs Whisper Tiny for live speech captions and MobileNet V2 for scene description, and renders both as real text in the DOM — announced through a polite live region, keyboard-operable, and verified by 11 automated WCAG 2.2 AA gates and 134 tests. We audited two live on-device-AI pages and found 30 accessibility issues between them, including 9 blockers; every category we found is now a regression test in ours. Three interchangeable backends — CPU, demo, and a Qualcomm AI Hub / QNN NPU path — sit behind one interface, so moving to a Snapdragon NPU is a configuration change rather than a rewrite. CPU latency is measured and published as a range; no NPU figure is claimed, because no Snapdragon device was available.

The problem

On-device AI demos are inaccessible by default. This is not a rare edge case — it is the normal outcome of building a visual demo.

- **Live captions that nobody hears.** Text that updates silently is invisible to a screen reader unless it sits in a live region.
- **A canvas is not an interface.** Scene descriptions drawn into a canvas have no text alternative at all.
- **A div is not a button.** A click handler does not give an element a role, an accessible name, a focus stop, or keyboard activation.
- **Status for everyone except the person who needs it.** Errors and confirmations that only appear visually.
- **Focus removed instead of replaced.** Suppressing the focus ring is a WCAG 2.4.7 failure that a screenshot-based review will not catch.

We measured it rather than assuming it

Two WCAG 2.2 AA audits of live on-device-AI pages, by rendered-DOM inspection, computed styles and real keyboard traversal: **30 issues in total** — 9 Blocker, 5 Serious, 11 Moderate, 5 Minor. Both audits are in the repository, and every category found is now a regression test in ours.

What it does

Live speech captions

Whisper Tiny (EN) — a 5 s rolling window, greedy decode, KV-cached. Announced through a polite live region, with the full transcript kept so it can be re-read.

Scene description

MobileNet V2 (ImageNet-1k) — top-3 classes with confidences, as real text in the DOM rather than drawn into a canvas. Works on a still image, a file, or a live camera.

No account. No network call. No data leaves the device. The app also runs with no model files at all — and says so, rather than faking output.

Measured results

On an Apple Silicon laptop running ONNX Runtime on **CPU** — **not an NPU**. Reproduce with `scripts/benchmark.py --backend onnx`.

Stage	Configuration	Result
Live speech-to-text	Whisper Tiny (EN), 5 s window	190–400 ms median, 12–26× real time
Scene description	MobileNet V2 (ImageNet-1k), 640×480	~5.3 ms, ~189 fps
Test suite	clean clone, weights fetched not vendored	134 passing
Accessibility gates	scripts/verify_a11y.py	11/11 pass
Correctness, speech	known reference clip	transcript matches exactly
Correctness, vision	known reference photograph	top-3 classes match

The ASR range is published rather than the best single run. Two Python environments on this one machine — same `onnxruntime` wheel, same session options — measured 194 ms and 402 ms for identical audio and an identical transcript, so the benchmark records the numeric stack next to every number it reports. What survives the variation is the claim that matters: comfortably faster than real time in every environment tested, with 12× as the floor.

What is verified, and what is not

No Snapdragon device was available during this build. The NPU backend is implemented against the real QNN and AI Hub APIs and documented step by step — but implemented is not measured, and no NPU figure is claimed anywhere in this project.

VERIFIED

- ✓ Both models run, and their output is checked against known-correct answers
- ✓ The full test suite passes from a clean clone (134 tests)
- ✓ The web app, its SSE caption stream, and the accessible UI
- ✓ 11/11 automated accessibility gates, plus a screen-reader pass
- ✓ Latency, measured on real audio and published as a range
- ✓ A clean-clone install with no model weights in the repository

NOT VERIFIED — no device

- ✗ Any Snapdragon NPU latency figure
- ✗ Any on-device photograph or screen capture
- ✗ QNN context-binary generation
- ✗ Qualcomm AI Hub compile and profile runs
- ✗ The Windows / DirectML execution path

Long description

The problem. On-device AI demos are inaccessible by default, and it is not a rare edge case — it is the normal outcome of building a visual demo. Live captions that update silently are invisible to a screen reader. Scene descriptions drawn into a canvas have no text alternative at all. A `div` with a click handler is not a button: it has no role, no accessible name, no focus stop and no keyboard activation. We did not assume this — we audited two live on-device-AI pages against WCAG 2.2 AA using rendered-DOM inspection, computed styles and real keyboard traversal, and recorded **30 issues, 9 of them blockers**: zero heading structure, a primary journey with no focusable elements, a closed dialog that was not inert, unnamed controls, and no skip link.

What we built. Snapdragoon is an offline, on-device accessibility co-pilot. Whisper Tiny produces live speech captions; MobileNet V2 describes what the camera sees with confidence scores. Both are rendered as real text in the DOM, announced through a polite live region so they never interrupt, with the full transcript kept so it can be re-read. The skip link is the first focusable element and its landing target keeps its focus ring. No `outline: none` appears anywhere in the codebase, and a test fails if it ever does.

Why it is built this way. The app never imports a runtime; it talks to an Engine interface with three backends — a self-labelling demo engine, a verified CPU engine, and a Qualcomm AI Hub / QNN NPU engine that raises rather than silently falling back when off-Snapdragon. That is what allowed the project to be built and verified without the target hardware, and it means a Snapdragon port is configuration, not a rewrite.

What is measured, and what is not. 134 tests pass from a clean clone. The reference clip transcribes exactly; the reference photograph classifies correctly. ASR runs a 5 s window in 190–400 ms (12–26× real time) and vision in ~5.3 ms (~189 fps) on a laptop CPU — published as a range, because two Python environments on the same machine differed by 2×. The Snapdragon NPU path is implemented and documented step by step but has **never been run on hardware**, so no NPU latency figure is claimed anywhere.

Claims deliberately excluded

Each of these was considered and rejected. They are listed so the reasoning survives whoever edits this document next.

Do not write	Why
“Runs on the NPU” / “NPU-accelerated”	Never measured. QualcommEngine raises off-Snapdragon precisely so this cannot be implied.
Any Snapdragon latency number	None exists. <code>scripts/benchmark.py --backend qualcomm</code> would produce one on the hardware.
“Tested on Snapdragon X”	No Snapdragon device was available during the build.
“13.2× real time”	Measured on silence, which the engine declines to transcribe. The corrected figure is on real audio.
A single ASR millisecond figure	It varies 2× with the Python environment. The range is the honest claim.

Run it

```
# git clone https://github.com/challalokesh08/snapdragoon
# cd snapdragoon && python -m venv .venv && source .venv/bin/activate
# pip install -r requirements.txt # runs with no model files
# python -m snapdragoon.web.app # → http://127.0.0.1:8765
```

With real inference: `pip install -r requirements-ml.txt && python scripts/fetch_models.py --public`

Models are fetched, never vendored — the repository stays small and carries no third-party weights.

Snapdragoon is an independent student project and is not affiliated with or endorsed by Qualcomm. “Snapdragon” is a trademark of Qualcomm Incorporated, used here to describe the target platform. Figures in this document are generated from the repository by `scripts/make_description_doc.py`.